MFASHA: A Kinyarwanda Agriculture Chatbot with Text and Voice Support

Technical Report: Final Version of the Product
Carine Ahishakiye Yibukabayo
BSc. in Software Engineering, African Leadership University
Supervisor: Hubert Apana
30 October 2026
Repository: https://github.com/carine-ahishakiye/Mfasha-Chatbot-.git

1. Introduction

MFASHA lets a farmer ask an agricultural question in Kinyarwanda, typed or recorded in the browser, and get a written answer that can also be played aloud. The answers are not generated. Each one is a passage taken unchanged from a Kinyarwanda agricultural collection, and when no passage matches well enough the app returns a referral message instead of a guess. I chose retrieval over generation because machine-written agricultural advice needs expert checking before farmers can rely on it (Ibrahim et al., 2024). The product is a FastAPI backend with a plain HTML, CSS and JavaScript interface written in Kinyarwanda. This report covers scope alignment, the technical implementation, testing, performance and compatibility, and code quality.

2. Scope Alignment and Feature Validation

The proposal listed candidate components and stated that any of them could be adopted, replaced or dropped depending on the evidence. Table 1 compares the plan with the repository.

Planned feature	Intended purpose	Status	Evidence
Typed question and written answer	Basic question answering	Completed	POST /api/ask, mfasha/pipeline.py, tests/test_api.py
Voice question recorded in browser	Farmers who find typing hard	Completed	chat.js recorder, POST /api/transcribe, mfasha/audio.py
Kinyarwanda speech-to-text, assessed with WER	Turn recording into text	Completed. Model not named in proposal (MMS-1b-all with Kinyarwanda adapter, fine-tuned)	mfasha/asr.py, results/asr_*.csv, WER 0.277
Text preprocessing	Consistent input	Completed	mfasha/text_utils.py
Question classification (KinyaBERT vs TF-IDF + LR/SVM)	Narrow search, detect out-of-scope	Not implemented. No topic labels or classifier stage; out-of-scope questions are caught by the retrieval threshold only	mfasha/pipeline.py

Retrieval and ranking against baselines	Find the right passage	Completed with a different candidate set. kiny-colbert-free and TF-IDF baseline not run; BM25, four embedding models and hybrids compared, winner fine-tuned	notebook/, results/retrieval_*.csv, results/plots/
Verified answer, confidence threshold, referral	Avoid wrong answers	Partial. Threshold and referral work. Officer review not done: review sheet has 941 passages with a risk level and no officer decisions	results/referral_threshold.json, results/passages_for_officer_review.xlsx
Text-to-speech with kinya-flex-tts	Spoken answers	Completed with a different model (facebook/mms-tts-kin). kinya-flex-tts not evaluated; a fine-tuned VITS voice was tried and not adopted	mfasha/tts.py, notebook TTS cells
Simple web interface, written and spoken answer	Usable with limited technical experience	Completed. FastAPI with HTML/CSS/JS instead of Streamlit	mfasha/web/
User feedback	Measure usefulness	Completed in simpler form. Votes appended to JSON Lines	POST /api/feedback, results/feedback.jsonl
PostgreSQL storage of questions, answers, model versions	Evaluation data	Not implemented. Only feedback votes are stored, in a JSON Lines file	mfasha/web/routes.py

Table 1. Planned features against implementation status.

Justification of changes. The prepared data has no topic labels, so there is no classifier stage. Out-of-scope questions are handled by the retrieval threshold, and in the hand check all three off-topic questions were referred. Off-the-shelf embedding models scored below BM25 on Kinyarwanda, so the retrieval work went into fine-tuning the strongest model instead of adding more candidates. The stock TTS voice scored better than my fine-tuned one, so the stock voice is used. The proposal named Streamlit and PostgreSQL; the product uses FastAPI with plain HTML, CSS and JavaScript, and stores only feedback votes in a JSON Lines file. The pilot with farmers and officers and the officer review of passages are not part of this repository, and I list both as limitations in Section 8.

3. Technical Implementation and Core Functionality

Request flow. A typed question goes straight to cleaning. A recorded question is uploaded to POST /api/transcribe, decoded to 16 kHz mono with PyAV (soundfile as fallback for plain WAV), and transcribed by facebook/mms-1b-all with the Kinyarwanda adapter and my fine-tuned weights. The

clean text is embedded with fine-tuned multilingual-e5-large and compared by cosine similarity with 941 pre-computed passage embeddings. If the best score reaches the threshold, the top passage is returned unchanged with a confidence level (high, medium or low) and two related passages. Otherwise the farmer receives the referral message. The farmer can then ask for the answer to be read aloud through POST /api/speak.

Front-end and backend interaction. The browser records with MediaRecorder, sends raw audio, and calls the JSON endpoints /api/ask, /api/transcribe, /api/speak and /api/feedback. Conversation history stays in the browser's localStorage and the server keeps no recordings. Route handlers only validate input, call a service and shape the JSON.

Data handling. The 984 passages from kinya-ag-retrieval were reduced to 941 after 43 heading-only entries were dropped. Splits were built so that no question is shared with training, and the unseen splits share no passage with training; the notebook checks this with asserts. Speech test sentences were removed from all training and tuning sets.

Input limits and error handling. Questions are capped at 500 characters, uploads at 20 MB, recordings at 30 s. A sliding-window limiter allows 20 questions, 8 recordings and 40 speech requests per visitor per minute. A crashing model returns a friendly error, and empty, oversized or broken input is rejected.

4. Algorithms and Technical Problem-Solving

- Hard-negative fine-tuning of the retriever. I used CachedMultipleNegativesRankingLoss. For each training question, one wrong passage drawn from the BM25 results at ranks 2 to 15 (restricted to passages that appear as training positives) was added as an extra negative. The 250k-row word embedding table was frozen so training fits on a 15 GB T4.
- Data-driven referral threshold. The threshold is the 5th percentile of the best-passage score over correctly answered dev questions, so 95% of correct retrievals are kept. It is 0.5723 from 2,693 dev questions and is stored in results/referral_threshold.json, not typed in by hand.
- Confidence level. mfasha/confidence.py grades each answer from the best score and its gap to the second-best score.
- Number-to-words for speech. The voice model cannot read digits, so mfasha/kin_numbers.py writes numbers, percentages, units, ranges and decimals as Kinyarwanda words before synthesis. • Spoken-answer handling. Answers are cut at a sentence or clause boundary within 250 characters and synthesised one clause at a time with pauses. Voices are cached and started in the background while the farmer reads, and if two requests ask for the same voice the second waits instead of generating it again.

5. Testing and Validation

I used four approaches. Each is described with the reason for choosing it, how it was run, the result, and what changed afterwards.

5.1 Unit and functional testing

Why: the answer-or-refer logic and the helper modules decide what a farmer sees, so they need to be checked apart from the large models. How: pytest with small fake models, so nothing is downloaded. Tests cover the answer, refer and empty cases, a score exactly at the threshold, threshold 0, threshold read from config and from file, confidence levels (including a single result and missing cutoffs), number and text conversion, speech-text cutting, the voice cache and its size limit, and the rate limiter being freed after its time window. Result: all 42 tests pass in about a second.

5.2 Integration testing

Why: the value of the product is in the endpoints working together with error handling. How:

FastAPI's test client runs the real application code against injected fake models. Cases include valid, empty, too long and missing-field questions, a crashing model, the 21st question hitting the rate limit, broken and empty audio, empty text to speak, feedback saving and bad votes, and search speed. Result: every endpoint behaves as specified. The tests were possible because the pipeline and create_app() accept their dependencies through the constructor. 5.3 Held-out model evaluation Why: scores measured on training data would mislead. How: retrieval was scored on *test_seen* (new questions about passages used in training) and *test_unseen* (whole passages never seen). Speech recognition was scored on 200 sentences (19 minutes) from one voice kept out of training and tuning. Results are in Tables 2 and 3.

System	seen R@1	seen MRR@10	unseen R@1	unseen MRR@10
BM25 (baseline)	0.487	0.595	0.489	0.599
paraphrase-multilingual-mpnet	0.032	0.054	0.046	0.069
LaBSE	0.082	0.133	0.085	0.121
bge-m3	0.348	0.446	0.328	0.427
multilingual-e5-large, not fine-tuned	0.400	0.491	0.316	0.396
BM25 + multilingual-e5-large	0.517	0.629	0.468	0.595
multilingual-e5-large, fine-tuned (used)	0.893	0.936	0.866	0.923

Table 2. Retrieval results on the test splits.

For the fine-tuned model R@5 is 0.990 on test_seen and 0.988 on test_unseen, and R@10 is 0.994 on both. Dev R@1 on the seen split over three epochs was 0.886, 0.912 and 0.912, so later epochs added little. Most errors come from passages on nearly the same topic, such as two passages on orange tree pruning. The referral threshold on 2,693 dev questions answers 90.5% of them, 91.1% of the answered ones have the right passage, and 139 of 356 wrong-top-passage questions were referred instead. In a hand check, three off-topic questions were referred (best scores 0.19 to 0.34) and six farming questions scored 0.60 to 0.82.

Speech-to-text (200 test sentences)	WER	WER no apostrophes	CER
MMS-1b-all, Kinyarwanda adapter as published	0.301	0.293	0.063
Same adapter, fine-tuned in this project	0.277	0.269	0.058

Table 3. Speech recognition results.

Improvements made afterwards: the weak off-the-shelf models led me to fine-tune the retriever (R@1 on test_seen from 0.400 to 0.893). The referral threshold was set from dev data instead of by

hand. For TTS, the fine-tuned VITS voice scored a word error rate of 0.695 when transcribed back, against 0.228 for the stock voice (recorded in mfasha/config.py), so I kept the stock voice.

5.4 Manual checks

Nine questions in results/ask_samples.csv (six farming, three off-topic) were run by hand. The file eval/off_topic_rw.txt holds 18 off-topic questions for a larger run of scripts/evaluate_answers.py. Native-speaker ratings of the synthesised voice were not collected.

6. Performance and Compatibility

Measured. On a Colab Tesla T4 (15 GB), the nine sample questions were answered in 0.06 to 0.09 s each (embedding, ranking and the threshold check). Transcribing the 200 test clips with the published speech model took 195 s. The app uses a GPU when present, and MFASHA_FAST_ASR=1 turns on int8 quantisation of the recogniser on CPU. The first start downloads about 6 GB (estimated in DEPLOY.md).

Not yet measured. I have not tested browsers or operating systems systematically, and I have not run the CPU against GPU comparison. scripts/benchmark_latency.py is written for this and records a hardware description with each timing, but no output from it is stored in the repository, so I report no figures for those environments. Voice questions are expected to be much slower without a GPU, and typed questions stay quick.

7. Code Quality and Architecture

The code is split so that each part can be tested and replaced on its own, and there is no single monolithic script.

- Abstraction and polymorphism. mfasha/interfaces.py defines three abstract classes: PassageSearcher, SpeechRecognizer and SpeechSynthesizer. The retriever, the transcriber and the speaker implement them, so a model can be swapped without touching the pipeline.
- Dependency injection. The Mfasha pipeline receives its searcher and recogniser through its constructor and create_app() takes optional ready-made objects. This lets the tests run the real application with fake models.
- Separation of concerns and single responsibility. Confidence rules, rate limiting, audio decoding, number conversion and asset download each have their own module. The web layer is thin and shared objects live in one Services dataclass built per application.
- Encapsulation and concurrency. Locks in services.py and voice.py allow one speech recognition and one speech synthesis request at a time.
- Configuration in one place. Paths, model names and limits are in mfasha/config.py and can be overridden through environment variables.
- Organisation. app.py is the entry point, the mfasha package holds the logic, with scripts/, tests/, notebook/, data/, results/ and eval/ kept separate. Evaluation metrics and bootstrap intervals sit in mfasha/evaluation.py so the scripts reuse them.

8. Conclusion

In summary, the core path of typed or spoken question, retrieval, threshold-based referral and spoken answer is implemented and tested. The fine-tuned retriever reaches R@1 of 0.893 on seen and 0.866 on unseen passages, against 0.487 and 0.489 for BM25. The next steps are officer review of the 941 passages, benchmarking on CPU and across browsers, and a pilot with farmers.